

Approach: Alemeno Custom Marker Detector

1. Marker Selection

Between the two options provided, I selected **Marker 1**.

- It is a 140x140mm square with a solid thick black border.
- It has a single 20x20mm solid black square in the top-left corner, acting as an orientation indicator.
- The thick solid border makes it highly reliable to detect using contour-based computer vision approaches compared to the dashed border in Marker 2.
- It easily meets the >60% empty space requirement.

2. Core Architecture

The solution is built as a pure **React Native** application using **react-native-vision-camera** (v5) for high-performance camera access.

To achieve the "speed" evaluation criteria (< 3000ms scan-to-result), I opted to run the detection logic **entirely in JavaScript via Worklets**, utilizing VisionCamera's `useFrameOutput`. This allows the detection logic to run synchronously on a dedicated native frame thread without crossing the React Native bridge, achieving near-native performance while keeping the codebase strictly React Native / JS.

3. Detection Pipeline

The detection pipeline runs continuously at a throttled rate (approx 5 frames per second) to preserve battery while ensuring immediate response when a marker enters the frame.

Step 1: Grayscale & Downsampling

VisionCamera delivers raw frames (typically YUV). We extract the **Y-plane** directly, which naturally provides a grayscale (luminance) image. To massively speed up processing, we downsample the frame to a maximum dimension of 600px.

Step 2: Binarization

The grayscale image is converted to pure black and white (binary) using **Otsu's Method**. Otsu automatically calculates the optimal threshold to separate the dark marker from the lighter background, making the app robust to varying lighting conditions.

Step 3: Connected Component Labelling

We find all continuous "dark" shapes (blobs) in the image using a fast 4-way flood-fill connected component algorithm.

Step 4: Geometric Filtering (Marker Identification)

We filter the detected blobs to find the marker's outer border:

1. **Size check:** Must be between 0.2% and 60% of the frame.
2. **Aspect ratio:** Must be roughly square (0.7 to 1.3).

Step 5: Orientation Detection

Once the outer square is found, we scan its four internal corners (based on the expected 10% border width and 14% corner square size).

- If a blob matching the size of the small 20x20mm square is found in one of the corners, we confirm it's a valid marker.
- The corner it's found in dictates the orientation (TopLeft = 0°, TopRight = 90°, BottomRight = 180°, BottomLeft = 270°).

Step 6: Full-Resolution Extraction

When the detection passes, we take the bounding box, scale its coordinates back up to the **full 2000px+ resolution**, and crop it tightly.

Step 7: Orientation Correction & Scaling

The cropped image is rotated based on the orientation detected in Step 5 so that the corner square is always positioned in the Top-Left. Finally, the image is bi-linearly scaled exactly to **300x300px**.

4. UI/UX

- **Camera Screen:** Shows a modern, animated "scanning" overlay. It flashes green when a marker is detected to provide immediate feedback.
- **Results Screen:** Displays a 4x5 grid of the 20 perfectly extracted, 300x300px orientation-corrected markers, along with stats (Total Time, Confidence).

Why this approach?

- **Speed:** By using pure Y-plane extraction and downsampling before the expensive connected-component phase, we get native-like frame rates.
- **Robustness:** Otsu thresholding handles dynamic lighting, and strict geometric checks prevent false positives.
- **Accuracy:** Extraction is performed on the un-downsampled high-res frame to guarantee zero pixelation and strict adherence to the 300x300 output requirement.